山东大学
SHANDONG UNIVERSITY

Typst template for Shandong University

Continuously Improving...

孙更欣

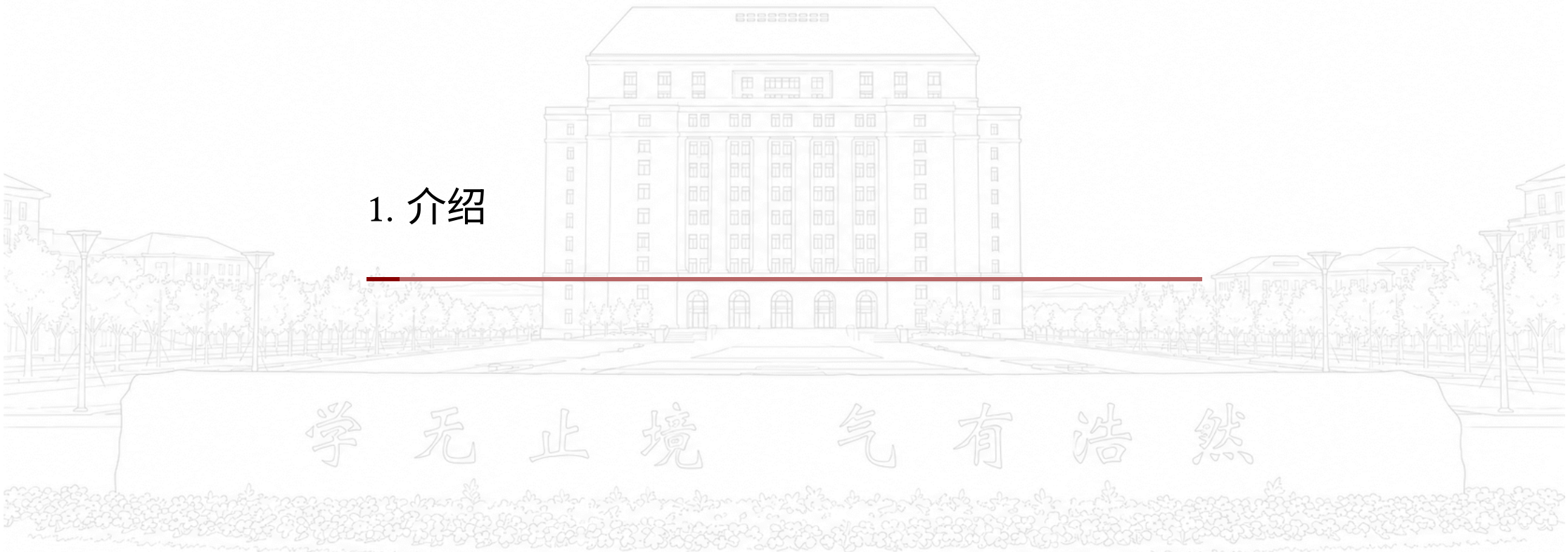
计算机科学与技术学院

2026-10-17

Knowledge Infinite• Spirit Vast
学无止境·气有浩然

Outline

- 介绍
- 安装
- 快速入门
- 部分基础功能展示
- 小组件
- 布局组件
- 页面



1. 介绍

什么是 Typst?

- 介绍：
  - Typst 是为写作而诞生的基于标记的排版系统。Typst 的目标是成为功能强大的排版工具，并且让用户可以愉快地使用它。

什么是 Typst?

- 介绍：
  - **Typst** 是为写作而诞生的基于标记的排版系统。**Typst** 的目标是成为功能强大的排版工具，并且让用户可以愉快地使用它。
- 简单来说：
  - **Typst** = LaTeX 的排版能力 + **Markdown** 的简洁语法 + 强大且现代的脚本语言

什么是 Typst?

- 介绍：
  - **Typst** 是为写作而诞生的基于标记的排版系统。**Typst** 的目标是成为功能强大的排版工具，并且让用户可以愉快地使用它。
- 简单来说：
  - **Typst** = LaTeX 的排版能力 + **Markdown** 的简洁语法 + 强大且现代的脚本语言
- 运行环境：Web Wasm / CLI / LSP Language Server
- 编辑器：Web App / VS Code / Neovim / Emacs

```
#set page(width: 10cm, height: auto)
#set heading(numbering: "1.")

= Fibonacci sequence
The Fibonacci sequence is defined through the recurrence relation $F_n = F_(n-1) + F_(n-2)$.
It can also be expressed in _closed form:_

$ F_n = round(1 / sqrt(5) phi.alt^n), quad phi.alt = (1 + sqrt(5)) / 2 $

#let count = 8
#let nums = range(1, count + 1)
#let fib(n) = (
  if n <= 2 { 1 }
  else { fib(n - 1) + fib(n - 2) }
)

The first #count numbers of the sequence are:

#align(center, table(
  columns: count,
  ..nums.map(n => $F_#n$),
  ..nums.map(n => str(fib(n))),
))
```

来源：Typst 官方 Repo

1. Fibonacci sequence

The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$. It can also be expressed in *closed form:*

$$F_n = \left\lfloor \frac{1}{\sqrt{5}} \phi^n \right\rceil, \quad \phi = \frac{1+\sqrt{5}}{2}$$

The first 8 numbers of the sequence are:

F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8
1	1	2	3	5	8	13	21

Typst 优势

- 语法简洁：上手难度跟 **Markdown** 相当，文本源码可阅读性高。

Typst 优势

- 语法简洁：上手难度跟 **Markdown** 相当，文本源码可阅读性高。
- 编译速度快：
  - Typst 使用 Rust 语言编写，即 `typ(esetting+ru)st`。
  - 增量编译时间一般维持在数毫秒到数十毫秒。

Typst 优势

- 语法简洁：上手难度跟 **Markdown** 相当，文本源码可阅读性高。
- 编译速度快：
  - Typst 使用 Rust 语言编写，即 `typ(esetting+ru)st`。
  - 增量编译时间一般维持在数毫秒到数十毫秒。
- 环境搭建简单：不像 LaTeX 安装起来困难重重，**Typst** 原生支持中日韩等非拉丁语言，官方 Web App 和本地 VS Code 均能开箱即用。

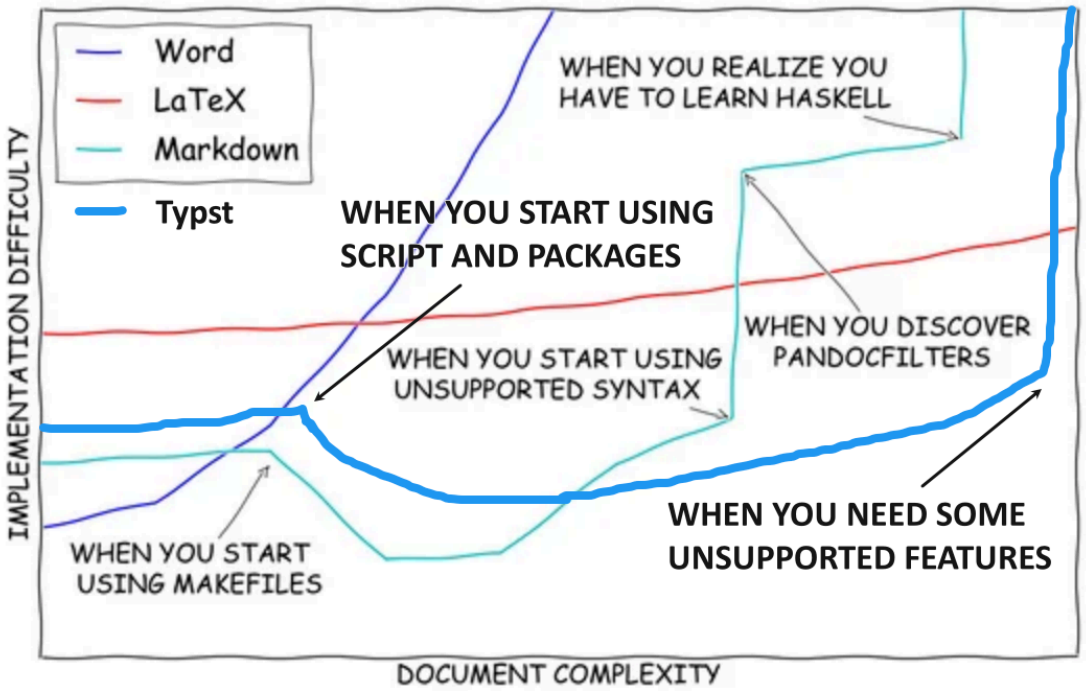
Typst 优势

- 语法简洁：上手难度跟 **Markdown** 相当，文本源码可阅读性高。
- 编译速度快：
  - Typst 使用 Rust 语言编写，即 `typ(esetting+ru)st`。
  - 增量编译时间一般维持在数毫秒到数十毫秒。
- 环境搭建简单：不像 LaTeX 安装起来困难重重，**Typst** 原生支持中日韩等非拉丁语言，官方 Web App 和本地 VS Code 均能开箱即用。
- 现代脚本语言：
  - 变量、函数、闭包与错误检查 + 函数式编程的纯函数理念
  - 可嵌套的 `[标记模式]`、`{脚本模式}` 与 `$ 数学模式 $` ~~不就是 JSX 嘛~~
  - 统一的包管理，支持导入 WASM 插件和按需自动安装第三方包

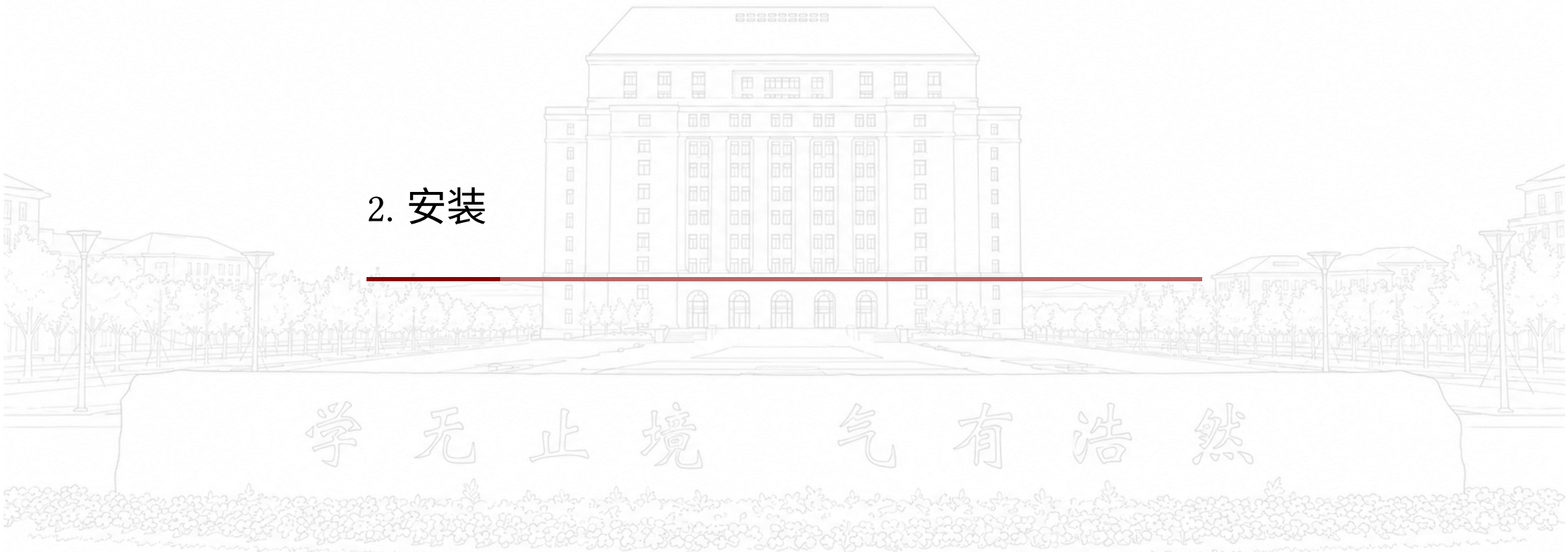
Typst 对比其他排版系统

排版系统	安装难度	语法难度	编译速度	排版能力	模板能力	编程能力	方言数量
LaTeX	难 选项多 + 体积大 + 流程复杂	难 语法繁琐 + 嵌套多 + 难调试	慢 宏语言编译 速度极慢	强 拥有最多的 历史积累	强 拥有众多的 模板和开发者	中等 图灵完备 但只是宏语言	中等 众多格式、 引擎和发行版
Markdown	易 大多编辑器 默认支持	易 入门语法十分简单	快 语法简单 编译速度较快	弱 基于 HTML 排版能力弱	中等 语法简单 易于更换模板	弱 图灵不完备 需要外部脚本	多 方言众多 且难以统一
Word	易 默认已安装	易 所见即所得	中等 能实时编辑 大文件会卡顿	强 大公司开发 通用排版软件	弱 二进制格式 难以自动化	弱 编程能力极弱	少 统一的标准和文件格式
Typst	易 安装简单 开箱即用	中等 入门语法简单 进阶使用略难	快 增量编译渲染 速度最快	较强 已满足日常 排版需求	强 制作和使用 模板都较简单	强 图灵完备 现代编程语言	少 统一的语法 统一的编译器

Typst 对比其他排版系统



From reddit r/LaTeX and modfied by OrangeX4



2. 安装

云端使用

- 官方提供了 Web App，可以直接在浏览器中使用

云端使用

- 官方提供了 Web App，可以直接在浏览器中使用
- 优点：
  - 即开即用，无需安装。
  - 类似于 LaTeX 的 Overleaf，可以直接编辑、编译、分享文档。
  - 拥有「多人协作」支持，可以实时共同编辑。

云端使用

- 官方提供了 Web App，可以直接在浏览器中使用
- 优点：
  - 即开即用，无需安装。
  - 类似于 LaTeX 的 Overleaf，可以直接编辑、编译、分享文档。
  - 拥有「多人协作」支持，可以实时共同编辑。
- 缺点：
  - 中文字体较少，经常需要手动上传字体文件，但有上传大小限制。
  - 缺少版本控制，目前无法与 GitHub 等代码托管平台对接。

本地使用（推荐）

- **VS Code 方案（推荐）**
  - 在插件市场安装「Tinymist Typst」插件。
  - 新建一个 `.typ` 文件，然后按下 Ctrl + K V 即可实时预览。
  - 不再需要其他配置，例如我们并不需要命令行安装 Typst CLI。

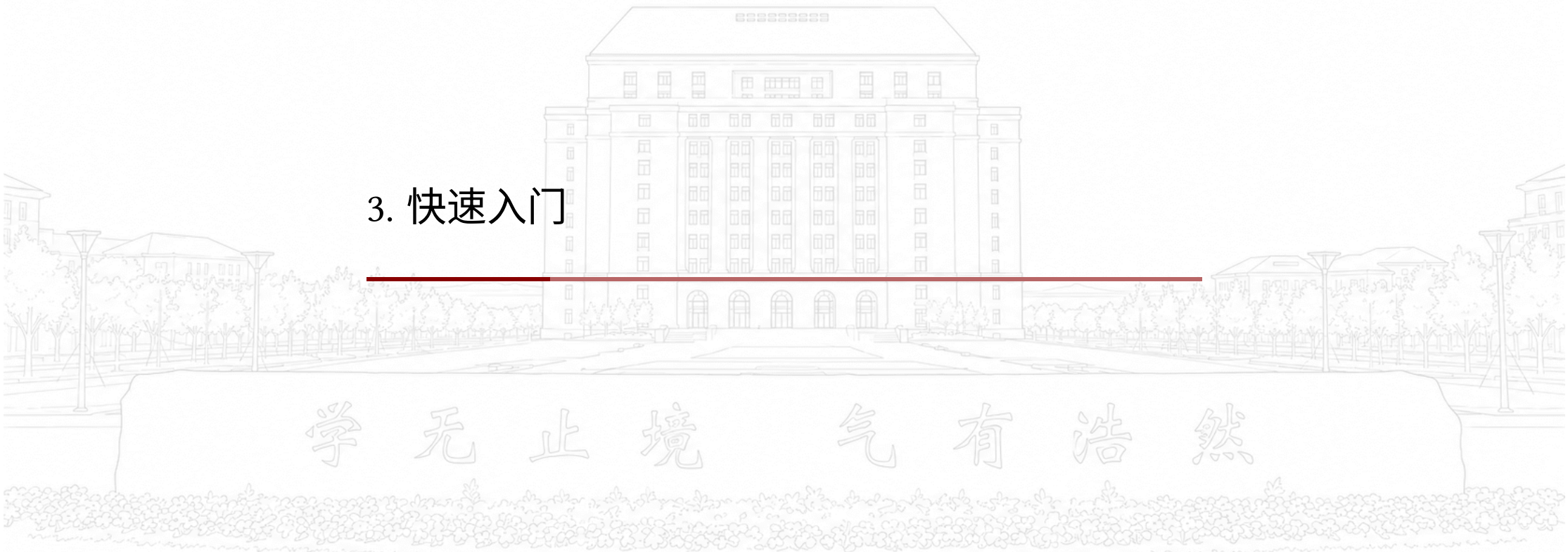
本地使用（推荐）

- **VS Code 方案（推荐）**
  - 在插件市场安装「Tinymist Typst」插件。
  - 新建一个 `.typ` 文件，然后按下 Ctrl + K V 即可实时预览。
  - 不再需要其他配置，例如我们并不需要命令行安装 Typst CLI。
- **Neovim / Emacs 方案**
  - 可以配置相应的 LSP 插件和 Preview 插件。

学 无 止 境　气 有 浩 然

本地使用（推荐）

- **VS Code 方案（推荐）**
  - 在插件市场安装「Tinymist Typst」插件。
  - 新建一个 `.typ` 文件，然后按下 Ctrl + K V 即可实时预览。
  - 不再需要其他配置，例如我们并不需要命令行安装 Typst CLI。
- **Neovim / Emacs 方案**
  - 可以配置相应的 LSP 插件和 Preview 插件。
- **CLI 方案：** `typst compile --root <DIR> <INPUT_FILE>`
  - Windows: `winget install --id Typst.Typst`
  - macOS: `brew install typst`
  - Linux：查看 Typst on Repology



3. 快速入门

Hello World

```typst
#set page(width: 20em, height: auto)
#show heading.where(level: 1): set align(center)

#show "Typst": set text(fill: blue, weight: "bold")
#show "LaTeX": set text(fill: red, weight: "bold")
#show "Markdown": set text(fill: purple, weight: "bold")

= Typst 讲座

Typst 是为 *学术写作* 而生的基于 _标记_ 的排版系统。

Typst = LaTeX 的排版能力 + Markdown 的简洁语法 + 现代的脚本语言

#underline[本讲座]包括以下内容：

+ 快速入门 Typst
+ Typst 编写各类模板
  - 笔记、论文、简历和 Slides
+ Typst 高级特性
  - 脚本、样式和包管理
+ Typst 周边生态开发体验
  - Pinit、MiTeX、Touying 和 VS Code 插件

```py
print('Hello Typst!')
```
```

Typst 讲座

Typst 是为 **学术写作** 而生的基于 *标记* 的排版系统。

Typst = **LaTeX** 的排版能力 + **Markdown** 的简洁语法 + 现代的脚本语言

本讲座包括以下内容：

1. 快速入门 **Typst**
2. **Typst** 编写各类模板
   - 笔记、论文、简历和 Slides
3. **Typst** 高级特性
   - 脚本、样式和包管理
4. **Typst** 周边生态开发体验
   - Pinit、MiTeX、Touying 和 VS Code 插件

```py
print('Hello Typst!')
```

标记只是语法糖

Typst

```
= 一级标题

== 二级标题

简单的段落，可以*加粗*和_强调_。

- 无序列表
+ 有序列表
/ 术语: 术语列表

```py
print('Hello Typst!')
```
```

Typst

```
#heading(level: 1, [一级标题])

#heading(level: 2, [二级标题])

简单的段落，可以#strong[加粗]和#emph[强调]。

#list.item[无序列表]
#enum.item[有序列表]
#terms.item[术语][术语列表]

#raw(lang: "py", block: true, "print('Hello Typst!')")
```

核心概念

- **Simplicity through Consistency**
  - 类似 **Markdown** 的特殊标记语法，实现「内容与格式分离」。
  - `= 一级标题` 只是 `#heading[一级标题]` 的语法糖。

核心概念

- **Simplicity through Consistency**
  - 类似 **Markdown** 的特殊标记语法，实现「内容与格式分离」。
  - `= 一级标题` 只是 `#heading[一级标题]` 的语法糖。
- 标记模式和脚本模式
  - 标记模式下，使用井号 `#` 进入脚本模式，如 `#strong[加粗]`。
    - 脚本模式下不需要额外井号，例如 `#heading(strong([加粗]))`
    - 大段脚本代码可以使用花括号 `{}`，例如 `#{1 + 1}`。

学 无 止 境　气 有 浩 然

核心概念

- **Simplicity through Consistency**
  - 类似 **Markdown** 的特殊标记语法，实现「内容与格式分离」。
  - `= 一级标题` 只是 `#heading[一级标题]` 的语法糖。
- 标记模式和脚本模式
  - 标记模式下，使用井号 `#` 进入脚本模式，如 `#strong[加粗]`。
    - 脚本模式下不需要额外井号，例如 `#heading(strong([加粗]))`
    - 大段脚本代码可以使用花括号 `{}`，例如 `#{1 + 1}`。
  - 脚本模式下，使用方括号 `[]` 进入标记模式，称为内容块。
    - **Typst** 是强类型语言，有常见的数据类型，如 `int` 和 `str`。
    - 内容块 `[]` 类型 `content` 是 **Typst** 的核心类型，可嵌套使用。
    - `#fn(..)[XXX][YYY]` 是 `#fn(.., [XXX], [YYY])` 的语法糖。

Set/Show Rules

- **Set 规则可以设置样式，即「为函数设置参数默认值」的能力。**
  - 例如 `#set heading(numbering: "1.")` 用于设置标题的编号。
  - 使得 `#heading[标题]` 变为 `#heading(numbering: "1.", [标题])`。

Set/Show Rules

- **Set 规则可以设置样式，即「为函数设置参数默认值」的能力。**
  - 例如 `#set heading(numbering: "1.")` 用于设置标题的编号。
  - 使得 `#heading[标题]` 变为 `#heading(numbering: "1.", [标题])`。
- **Show 规则用于全局替换，即在语法树上进行「宏编程」的能力。**
  - 例如 `#show "LaTeX": "Typst"` 将单词 LaTeX 替换为 Typst。
  - 例如让一级标题居中，可以用「箭头函数」：

```typst
#show heading.where(level: 1): body => {
  set align(center)
  body
}
```

    - 化简为 `#show heading.where(level: 1): set align(center)`

数学公式

- `$x$` 是行内公式，`$ x^2 + y^2 = 1 $ <circle>` 是行间公式。

数学公式

- `$x$` 是行内公式，`$ x^2 + y^2 = 1 $ <circle>` 是行间公式。
- 与 LaTeX 的差异：
  - `(x + 1) / x >= 1 => 1/x >= 0`
  - `\frac{x + 1}{x} \ge 1 \Rightarrow \frac{1}{x} \ge 0`

数学公式

- `$x$` 是行内公式，`$ x^2 + y^2 = 1 $ <circle>` 是行间公式。
- 与 LaTeX 的差异：
  - `(x + 1) / x >= 1 => 1/x >= 0`
  - `\frac{x + 1}{x} \ge 1 \Rightarrow \frac{1}{x} \ge 0`
- 报告，我想用 **LaTeX 语法**：

Typst

```
#import "@preview/mitex:0.2.2": *

Write inline equations like #mi("x") or #mi[y].
#mitex(`
  \frac{x + 1}{x} \ge 1 \Rightarrow \frac{1}{x} \ge 0
`)
```

Touying 对比其他 Slides 方案

方案	语法难度	编译速度	排版能力	模板能力	编程能力	动画效果	代码公式
PowerPoint	易 所见即所得	快 实时编辑	强 大公司开发 通用软件	强 模板数量最多 容易制作模板	弱 编程能力极弱 难以显示进度	强 动画效果多 但用起来复杂	难 难以插入代码和公式 贴图片
Beamer	难 语法繁琐 + 嵌套多 + 难调试	慢 宏语言编译 速度极慢	弱 使用模板后 排版难以修改	中等 拥有较多模板 开发模板较难	中等 图灵完备 但只是宏语言	中等 简单动画方便 无过渡动画	易 基本默认支持
Markdown	易 入门语法十分简单	快 语法简单 编译速度较快	弱 语法限制 排版能力弱	弱 难以制作模板 只有内置模板	弱 图灵不完备 需要外部脚本	中等 动画效果全看提供了什么	易 基本默认支持
Touying	易 语法简单 使用方便	快 增量编译渲染 速度最快	中等 满足日常学术 Slides 需求	强 制作和使用 模板都较简单	强 图灵完备 现代编程语言	中等 简单动画方便 无过渡动画	易 默认支持 MiTeX 包

4. 部分基础功能展示



注：这里也可以有内容

我是谁



我是基于 touying 制作的非官方 Typst 模板，用于展示山东大学的学术成果。



展示超长标题

想分列显示?

展示 `void fn`

Second column.第二列



标题

内容

列表

- 无序列表
- 第二项

1. 有序列表 1
2. 有序列表 2

表格

	Exam 1	Exam 2	Exam 3
John		A	
Mary		A	A
Robert	B	A	B

行内公式：$a^2 + b^2 = c^2$ 块级公式：

$$E = mc^2$$

$$\begin{aligned}\langle a, b\rangle &= \vec{a} \cdot \vec{b} \\ &= a_1 b_1 + a_2 b_2 + ...a_n b_n \\ &= \sum_{i=1}^{n} a_i b_i.\end{aligned}$$

Definition 4.8.1: A natural number is called a *prime number* if it is greater than 1 and cannot be written as the product of two smaller natural numbers.

Example: The numbers 2, 3, and 17 are prime. Corollary 4.8.1.1 shows that this list is not exhaustive!

Theorem 4.8.1 (Euclid): There are infinitely many primes.

Proof: Suppose to the contrary that $p_1, p_2, \ldots, p_n$ is a finite enumeration of all primes. Set $P = p_1 p_2 \ldots p_n$. Since $P + 1$ is not in our list, it cannot be prime. Thus, some prime factor p_j divides $P + 1$. Since p_j also divides P, it must divide the difference $(P + 1) - P = 1$, a contradiction. □

Corollary 4.8.1.1: There is no largest prime number.

Corollary 4.8.1.2: There are infinitely many composite numbers.

Theorem 4.8.2: There are arbitrarily long stretches of composite numbers.

Proof: For any $n > 2$, consider

$$n! + 2, \quad n! + 3, \quad \ldots, \quad n! + n$$

□

定理 1. 欧几里得
素数有无穷多个。

定义 2

一个大于 1 的自然数，若不能被更小的自然数乘积表示，则称为素数。

引理 3

任意两个素数互素。

推论 4

素数集合是无限集。

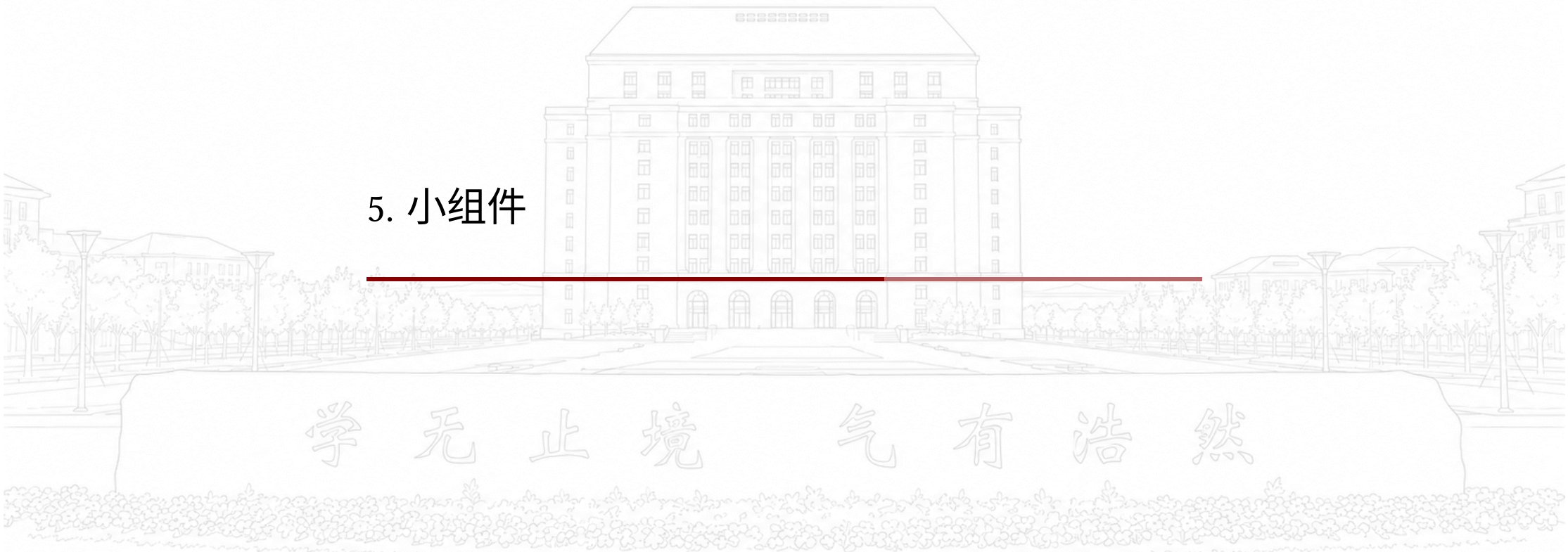
由定理 1 可知推论 1。

证明

假设素数有限，设为 $p_1, p_2, \ldots, p_n$。考虑 $P = p_1 p_2 \ldots p_n + 1$，则 P 不被任何 p_i 整除，矛盾。 □

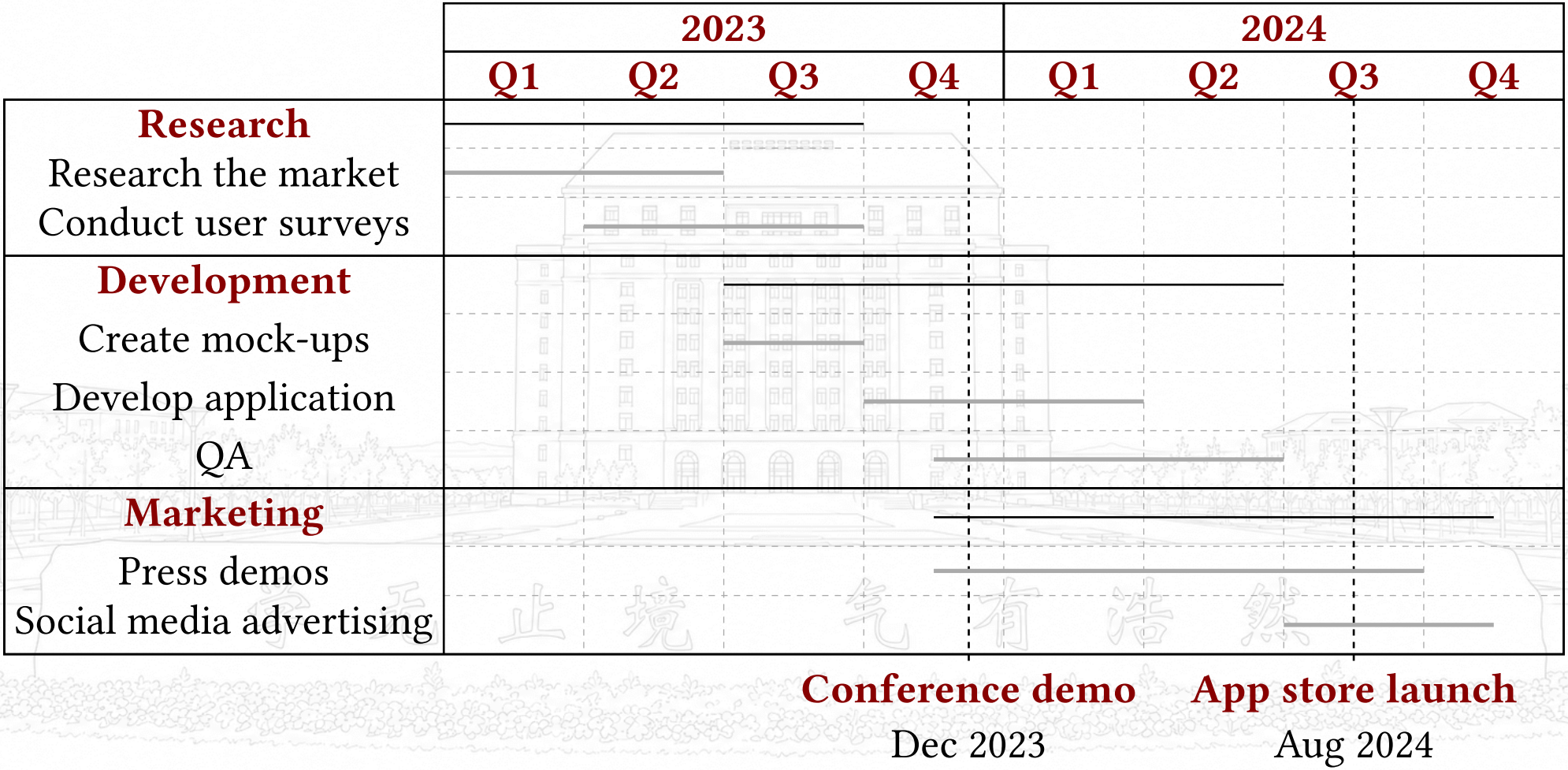
示例

最小的几个素数为 $2, 3, 5, 7, 11$。



5. 小组件

时间轴，很简单


2023
2024
Q1
Q2
Q3
Q4
Q1
Q2
Q3
Q4
Research
Research the market
Conduct user surveys
Development
Create mock-ups
Develop application
QA
Marketing
Press demos
Social media advertising
Conference demo
Dec 2023
App store launch
Aug 2024

代码块，很优雅

```
1 pub fn main() {
2     println!("Hello, world!");
3 }
```
Typst

```
1 pub fn main() {
2     println!("Hello, world!");
3 }
```
Rust

SDU 代码高亮

Python

```python
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)
```

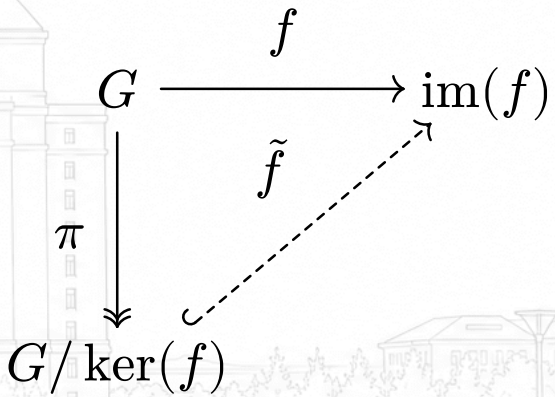
Rust

```rust
fn main() {
    let greeting = "Hello, SDU!";
    println!("{}", greeting);
}
```

用节点和箭头绘制图表

```
#diagram(cell-size: 15mm, $
  G edge(f, ->) edge("d", pi, ->>) & im(f) \
  G slash ker(f) edge("ur", tilde(f), "hook-->")
$)
```

Typst

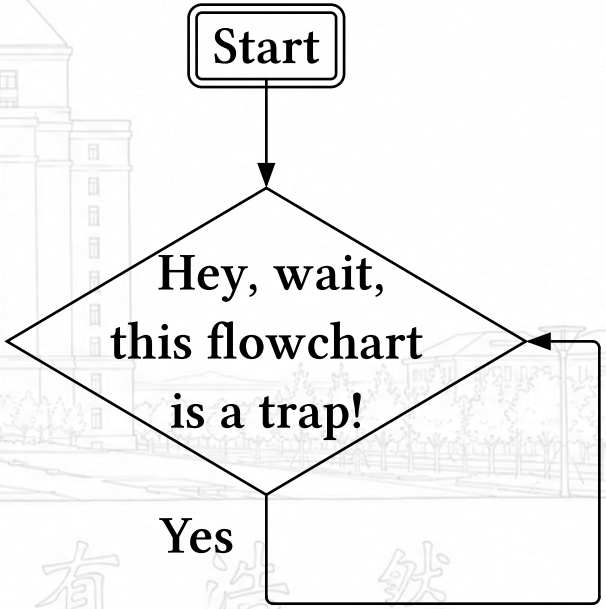


[1] https://typst.app/docs

用节点和箭头绘制图表

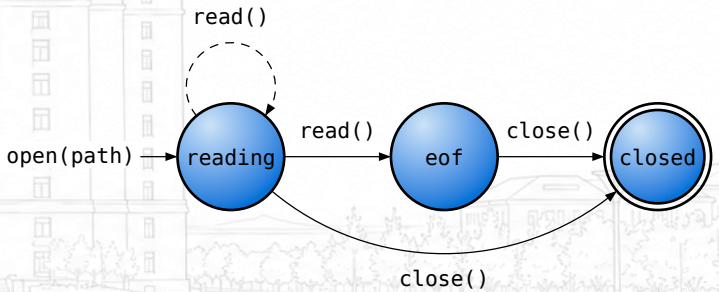
```typst
#import fletcher.shapes: diamond
#set text(font: "Comic Neue", weight: 600)

#diagram(
  node-stroke: 1pt,
  edge-stroke: 1pt,
  node((0,0), [Start], corner-radius: 2pt, extrude: (0, 3)),
  edge("-|>"),
  node((0,1), align(center)[
    Hey, wait,\ this flowchart\ is a trap!
  ], shape: diamond),
  edge("d,r,u,l", "-|>", [Yes], label-pos: 0.1)
)
```



用节点和箭头绘制图表

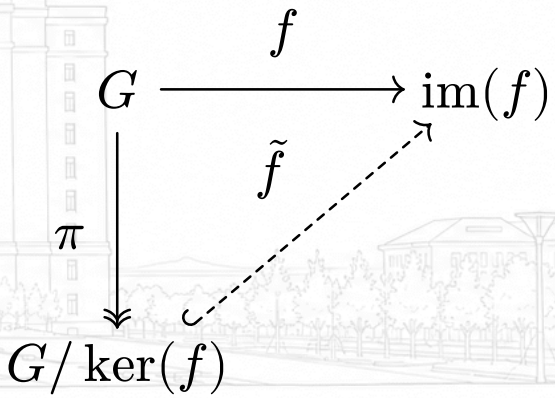
```
#set text(10pt)
#diagram(
  node-stroke: .1em,
  node-fill: gradient.radial(blue.lighten(80%), blue, center: (30%, 20%), radius: 80%),
  spacing: 4em,
  edge((-1,0), "r", "-|>", `open(path)`, label-pos: 0, label-side: center),
  node((0,0), `reading`, radius: 2em),
  edge(`read()`, "-|>"),
  node((1,0), `eof`, radius: 2em),
  edge(`close()`, "-|>"),
  node((2,0), `closed`, radius: 2em, extrude: (-2.5, 0)),
  edge((0,0), (0,0), `read()`, "--|>", bend: 130deg),
  edge((0,0), (2,0), `close()`, "-|>", bend: -40deg),
)
```



用节点和箭头绘制图表

```
#diagram(cell-size: 15mm, $
  G edge(f, ->) edge("d", pi, ->>) & im(f) \
  G slash ker(f) edge("ur", tilde(f), "hook-->")
$)
```

Typst



展示框，很有趣

```typst
#showybox(
  [Hello world!]
)
```

```typst
showybox(
  frame: (
    dash: "dashed",
    border-color: red.darken(40%)
  ),
  body-style: (
    align: center
  ),
  sep: (
    dash: "dashed"
  ),
  shadow: (
    offset: (x: 2pt, y: 3pt),
    color: yellow.lighten(70%)
  ),
  [This is an important message!],
  [Be careful outside. There are dangerous bananas!]
)
```

Hello world!

This is an important message!

Be careful outside. There are dangerous bananas!

提示框

```
#info[ This is the info clue ... ]
#tip(title: "Best tip ever")[Check out this cool package]
```

Typst

Info

This is the info clue ...

Best tip ever

Check out this cool package

类 obsidian

Info

This is information

Success

I'm making a note here: huge success

Warning

First warning...

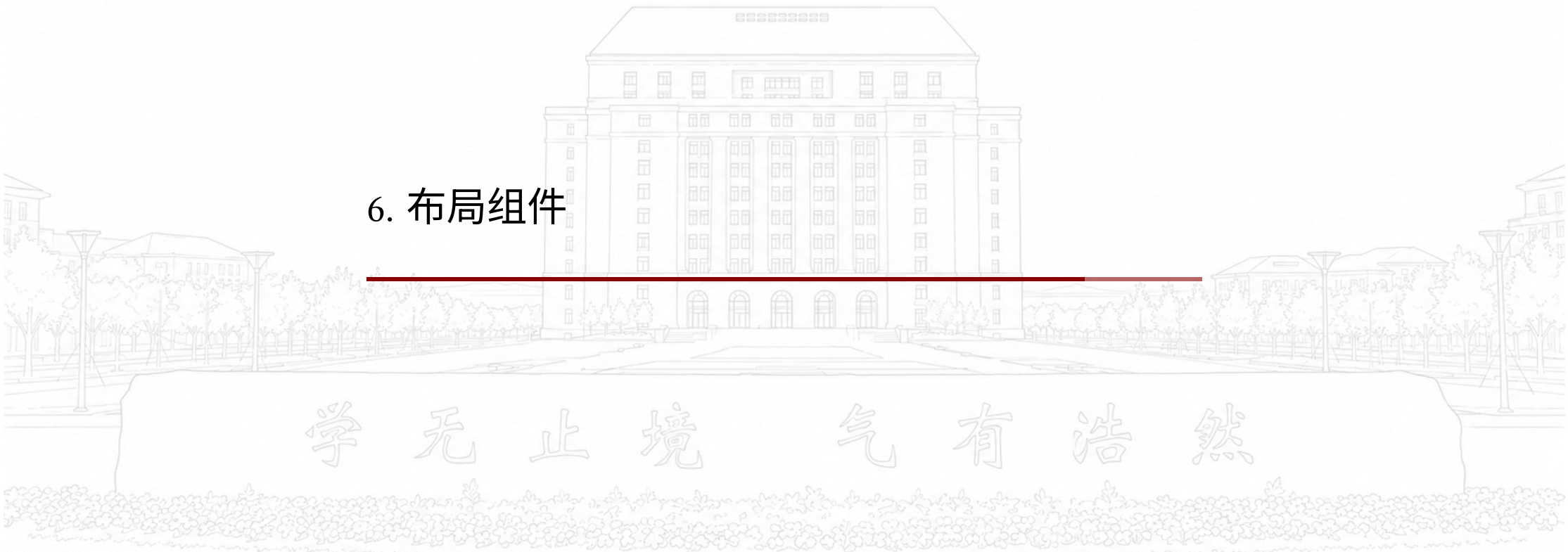
? Question

Question?



Example

An example make things interesting



6. 布局组件

多栏布局

第一栏 – 文字内容

第二栏 – 更多内容

第三栏 – 继续

性能对比

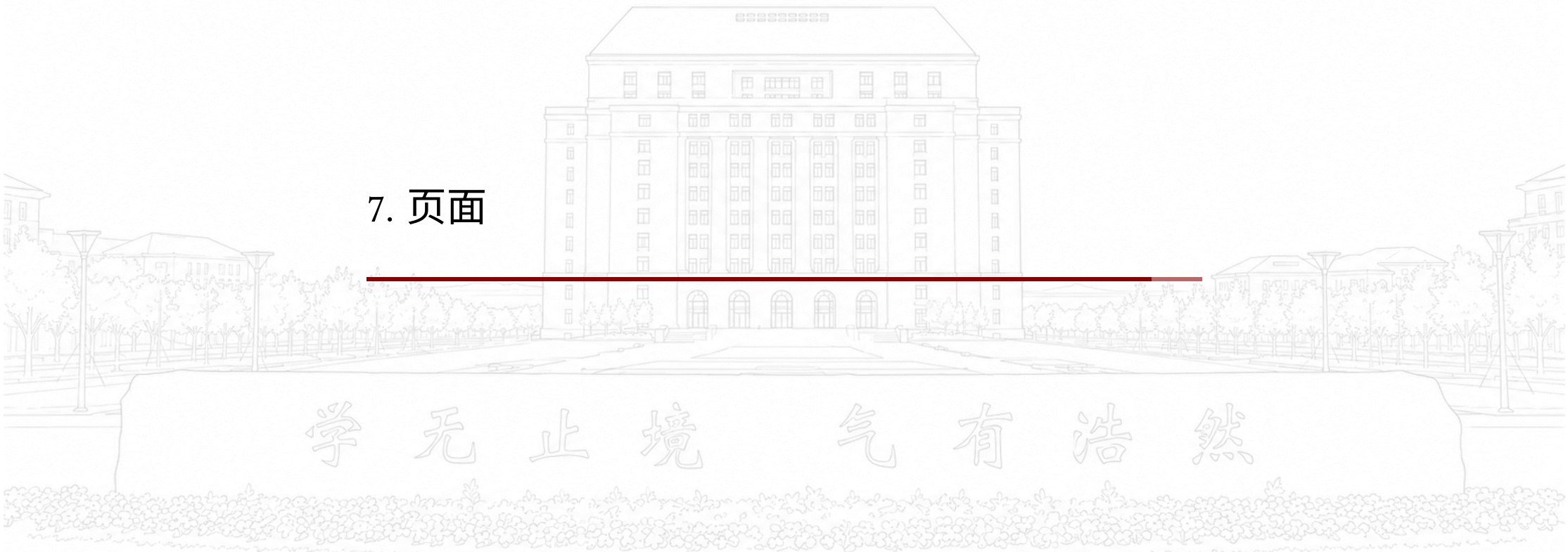
Typst 增量编译一般在数毫秒内完成。

结论： Typst 在编译速度上显著优于 LaTeX。

> 预测未来的最好方法是创造未来。
>
> — Donald Knuth

表格样式

排版系统	编译速度	语法难度	生态成熟度
LaTeX	慢	难	强
Typst	快	中	成长中
Word	中	易	强
Markdown	快	易	中



7. 页面

聚焦页

left

middle

right

top

bottom

Lorem ipsum dolor sit amet, consectetur adipiscing elit.	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Lorem ipsum dolor sit amet, consectetur adipiscing elit.	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Lorem ipsum dolor sit amet, consectetur adipiscing elit.	Lorem ipsum dolor sit amet, consectetur adipiscing elit.	Lorem ipsum dolor sit amet, consectetur adipiscing elit.

引用

引用如下。[1]

[1] I. Author, “The definition of importance The definition of importance The definition of importance The definition of importance The definition of importance The definition of importance,” *Journal on Important Stuff*, 1978.

致谢

1. Typst 官方文档
2. Typst 中文教程
3. Typst 非官方中文交流群 793548390
4. OrangeX4 的 Typst 讲座（几乎占了介绍内容的全部）
5. Touying 官方文档
6. 中科大 touying-pres-ustc 模板
7. touying-brandred-uobristol
8. sdu-touying-template

THANKS FOR ALL

敬请指正！